Military Asset Management System - Project Documentation

1. Project Overview

The **Military Asset Management System (MAMS)** is designed to serve as a comprehensive platform for military organizations to efficiently oversee their logistical operations. At its core, MAMS enables seamless tracking, management, and reporting of critical military assets such as vehicles, weapons, ammunition, and other essential equipment spread across multiple geographically distributed military bases.

The system provides **real-time transparency** into inventory levels, movements, and asset utilization, allowing commanders and logistics personnel to make informed decisions quickly. By centralizing asset data and automating movement and assignment tracking, MAMS reduces manual errors, improves accountability, and eliminates redundant paperwork traditionally involved in logistics management.

With **secure role-based access control (RBAC)**, MAMS ensures that only authorized personnel can access or modify sensitive data based on their assigned roles—whether administrators, base commanders, or logistics officers—thereby maintaining strict data confidentiality and operational security.

The system incorporates **detailed audit logging**, which records every transaction involving purchases, transfers, assignments, and expenditures. This feature is crucial not only for accountability but also for post-operation audits, compliance reporting, and strategic reviews. The detailed logs ensure full traceability of asset movements, preventing loss, misuse, or misallocation of critical resources.

MAMS has been designed with **modular scalability**, allowing it to adapt as operational needs grow. Whether deployed at a single base or across an entire military region, the system maintains consistent performance while ensuring data integrity across the organization.

In addition, the system supports **streamlined workflows** that reduce delays in asset procurement, simplify inter-base transfers, and ensure that personnel are always equipped with the resources they need for their missions. By consolidating fragmented processes into a unified digital system, MAMS promotes operational efficiency, readiness, and mission success.

Key Features:

- Manage opening and closing balances of assets.
- Track purchases, transfers, assignments, and expenditures.
- Role-based access for administrators, base commanders, and logistics officers.
- Secure APIs with JWT-based authentication.
- Transaction logging for auditing and accountability.

- Simple and scalable backend architecture.
- Easy integration for future frontend development.
- Reliable MySQL relational database for consistent data storage.

Assumptions:

- Only authorized personnel can access the system.
- Each base operates independently but data is centrally managed.
- System is accessed via web and mobile devices over secure HTTPS connections.

Limitations:

- No real-time sync across disconnected environments.
- External integrations (ERP, accounting) are not included in current version.

2. Tech Stack & Architecture

Backend

The backend system forms the core of the Military Asset Management System. It is responsible for handling authentication, authorization, business logic, data storage, and retrieval.

Technologies Used:

- **Node.js with Express.js**
  - **Reason:** Node.js is asynchronous, non-blocking, and event-driven, making it highly efficient for scalable API development. Express.js simplifies the routing, middleware, and request-response handling in a clean and maintainable way.
  - **Code Example:**

```
javascript
CopyEdit
const express = require('express');
const app = express();
app.use(express.json());
app.get('/api/health', (req, res) => {
   res.send('API is running');
});
app.listen(3000, () => console.log('Server running on port 3000'));
```

MySQL (Relational Database)

  - **Reason:** MySQL provides full ACID compliance, transaction support, strong relational modeling, and excellent support for complex queries, joins, reporting, and aggregation, all critical for military asset tracking and auditing.

- **Example SQL Schema:**

```
sql
CopyEdit
CREATE TABLE Users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE,
  password VARCHAR(255),
  role ENUM('Admin','BaseCommander','LogisticsOfficer'),
  base_id INT NULL
);
```

JWT (JSON Web Tokens)

- **Reason:** Allows for stateless, secure authentication where each API request contains the signed token in headers, eliminating the need for server-side session storage.
- **Code Example:**

```
javascript
CopyEdit
const jwt = require('jsonwebtoken');
const token = jwt.sign({ userId: user.id, role: user.role }, process.env.JWT_SECRET, {
expiresIn: '1h' });
res.json({ token });
```

- **bcrypt.js**
  - **Reason:** Provides strong one-way password hashing, protecting user credentials against breaches.
  - **Code Example:**

```
javascript
CopyEdit
const bcrypt = require('bcrypt');
const hashedPassword = await bcrypt.hash(password, 10);
```

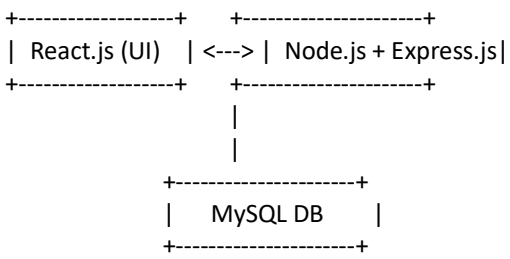
- **dotenv**
  - **Reason:** Manages environment-specific configurations securely via a .env file.
  - **Example .env File:**

```
ini
CopyEdit
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=military_asset_db
JWT_SECRET=super_secret
PORT=3000
```

Frontend

- While this phase focuses on backend API development, the backend has been fully designed with frontend integration in mind. The REST APIs are easily consumable by any frontend technology.
- **React.js** has been selected for future frontend development due to:
  - Component-based architecture
  - Fast state management with hooks
  - Easy integration with RESTful APIs

Architecture Diagram



Key Architecture Decisions

- **RESTful API Design**
  Every major business operation (Purchases, Transfers, Assignments, Expenditures) is implemented via clean RESTful endpoints.
- **Secure RBAC Middleware**
  Every route is protected based on user role using middleware logic:

```
javascript
CopyEdit
const authMiddleware = (roles) => (req, res, next) => {
  const token = req.headers.authorization;
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  if (!roles.includes(decoded.role)) return res.status(403).send('Access denied');
  next();
}
```

- **Centralized MySQL DB**
  All military bases share a centralized database, but access is restricted by base for Base Commanders and Logistics Officers.
- **API Logging System**
  Each API call that modifies data is logged with timestamp, user, and transaction data for audit compliance.

```
javascript
CopyEdit
const logTransaction = (userId, action, data) => {
```

```
    console.log(`[${new Date().toISOString()}] User:${userId} -> ${action}: ${JSON.stringify(data)}`);
}
```

3. Data Models / Schema

Tables and Core Relationships:

Users Table

- id (PK)
- username (unique)
- password (hashed)
- role (Admin, BaseCommander, LogisticsOfficer)
- base_id (nullable)

Bases Table

- id (PK)
- name
- location

EquipmentTypes Table

- id (PK)
- name

Assets Table

- id (PK)
- base_id (FK)
- equipment_type_id (FK)
- quantity (current balance)

Purchases Table

- id (PK)
- base_id (FK)
- equipment_type_id (FK)
- quantity
- purchase_date

Transfers Table

- id (PK)
- from_base_id (FK)

- to_base_id (FK)
- equipment_type_id (FK)
- quantity
- transfer_date

Assignments Table

- id (PK)
- base_id (FK)
- equipment_type_id (FK)
- quantity
- assigned_to (personnel name)
- assignment_date

Expenditures Table

- id (PK)
- base_id (FK)
- equipment_type_id (FK)
- quantity
- expended_date

4. RBAC Explanation (Role Based Access Control)

Roles Defined:

Admin:

- Full access to all data and operations.
- User management (register users).

BaseCommander:

- Access data for assigned base.
- Can assign assets to personnel.

LogisticsOfficer:

- Can create purchases and transfers for assigned base.
- Limited read access across assets.

Enforcement:

- RBAC implemented via middleware.
- Every protected API endpoint checks JWT token and verifies role.
- Unauthorized requests return HTTP 401/403 error.

5. API Logging

- All sensitive operations (purchases, transfers, assignments, expenditures) are logged.
- Log files include timestamp, user ID, operation type, and affected records.
- Logs can be extended to use centralized logging solutions like ELK stack or cloud-based monitoring.

Example log format:

[2024-06-25 10:30:15] UserID: 4 (LogisticsOfficer) -> TRANSFER {from_base: 1, to_base: 2, equipment: 'Ammunition', qty: 50}

Why Logging is Critical

In a military-grade application, logging is **not optional**—it's critical for:

- **Accountability:** Who did what, when, and where.
- **Auditing & Compliance:** Logs can be reviewed during military audits.
- **Security Investigations:** Helps detect misuse or unusual activity.
- **Debugging & Monitoring:** Identifies patterns, failures, and performance issues.

What We Log

Each sensitive transaction—**purchases, transfers, assignments, expenditures**—generates a log entry including:

- Timestamp
- User ID and Role
- Action type (e.g., PURCHASE, TRANSFER)
- Payload or affected values
- Status (optional: success, failure)

🗀 **Log Format**

Example output in console or file:

[2024-06-25 10:30:15] UserID: 4 (LogisticsOfficer) → TRANSFER { from_base: 1, to_base: 2, equipment: 'Ammunition', qty: 50 }

Implementation in Code (Express + Node.js)

✅ **Step 1: Create a logger utility**

utils/logger.js

```
js
CopyEdit
const fs = require('fs');
const path = require('path');

const logFile = path.join(__dirname, '../logs/api.log');

const logAction = (userId, role, actionType, data) => {
  const timestamp = new Date().toISOString();
  const logEntry = `[${timestamp}] UserID: ${userId} (${role}) → ${actionType} ${JSON.stringify(data)}\n`;

  // Log to console (optional)
  console.log(logEntry);

  // Append to log file
  fs.appendFile(logFile, logEntry, (err) => {
    if (err) console.error('Failed to write log:', err);
  });
};

module.exports = logAction;
```

Step 2: Use it inside a controller (e.g., Transfers API)

controllers/transferController.js

```
js
CopyEdit
const logAction = require('../utils/logger');

const createTransfer = async (req, res) => {
```

```
  try {
    const { from_base_id, to_base_id, equipment_type, quantity, transfer_date } = req.body;
    const user = req.user; // Assume user is decoded from JWT middleware

    // (Insert DB logic here to save transfer...)

    // Log the action
    logAction(user.id, user.role, 'TRANSFER', {
      from_base: from_base_id,
      to_base: to_base_id,
      equipment: equipment_type,
      qty: quantity
    });

    return res.status(200).json({ message: "Transfer recorded" });
  } catch (error) {
    return res.status(500).json({ message: "Internal error" });
  }
};
```

6. Setup Instructions

Backend Setup

1. Install Node.js (v16 or above).
2. Clone the repository.
3. Install dependencies:

```
npm install
```

4. Create a .env file:

```
DB_HOST=localhost
DB_USER=root
```

DB_PASSWORD=your_mysql_password

DB_NAME=military_asset_db

JWT_SECRET=your_secret_key

PORT=3000

5. Import MySQL schema (provided in schema.sql).
6. Run server:

npm start

MySQL Setup

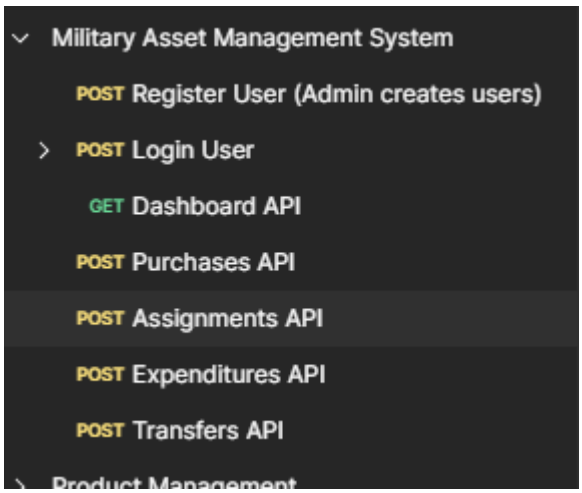
- Install MySQL Server.
- Create database military_asset_db.
- Use provided SQL script to create tables.

Frontend Setup

- This version focuses on backend APIs.
- Frontend can be easily built on top of the REST APIs using React or any frontend framework.

7. API Endpoints (Key Functional APIs)

Authentication APIs



Register User (Admin only)

POST /api/auth/register

{

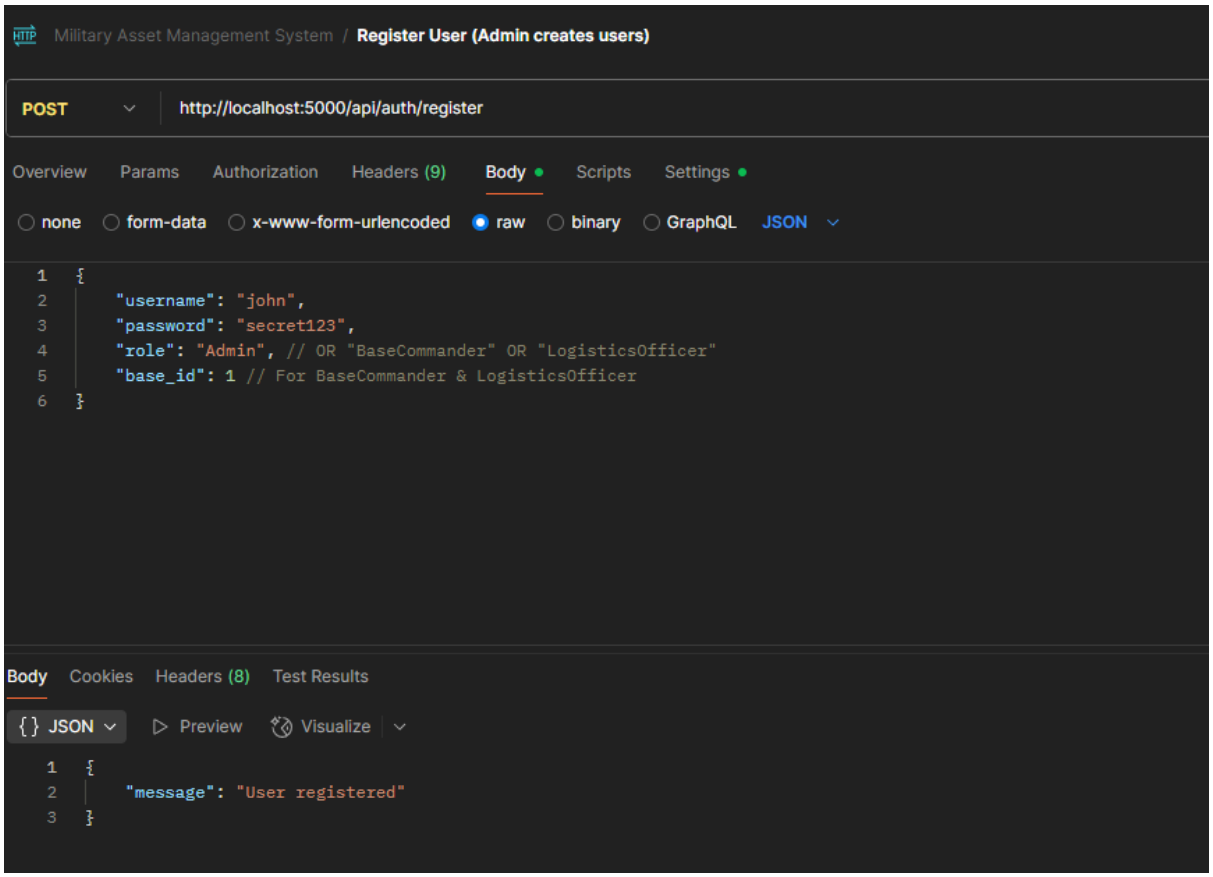
"username": "john",

"password": "secret123",

"role": "Admin", // OR "BaseCommander" OR "LogisticsOfficer"

"base_id": 1 // For BaseCommander & LogisticsOfficer

}



Login User

POST /api/auth/login

{

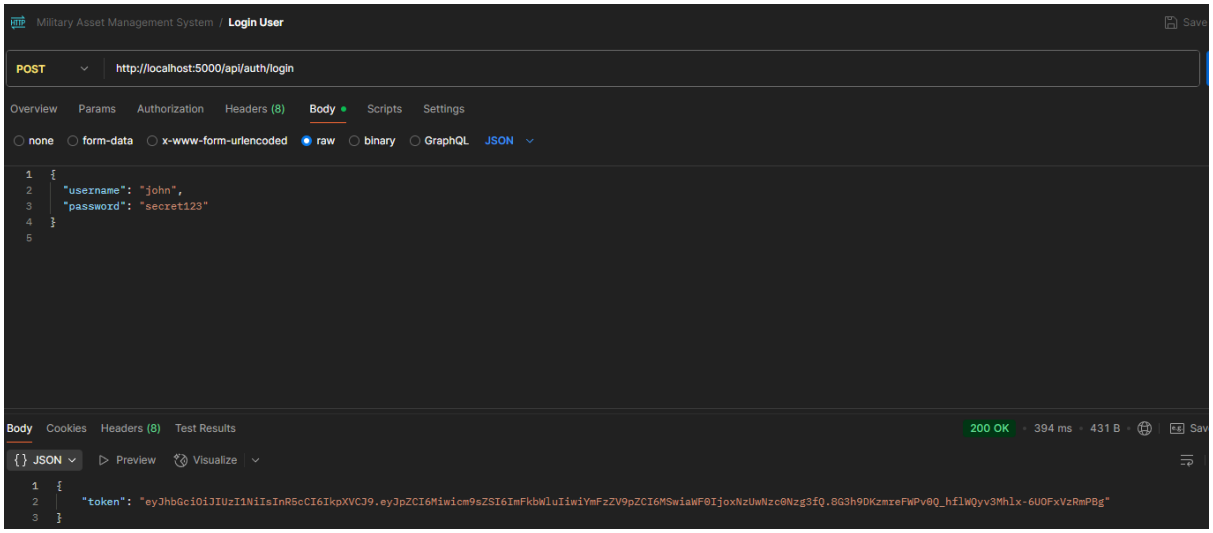
"username": "john",

"password": "secret123"

}

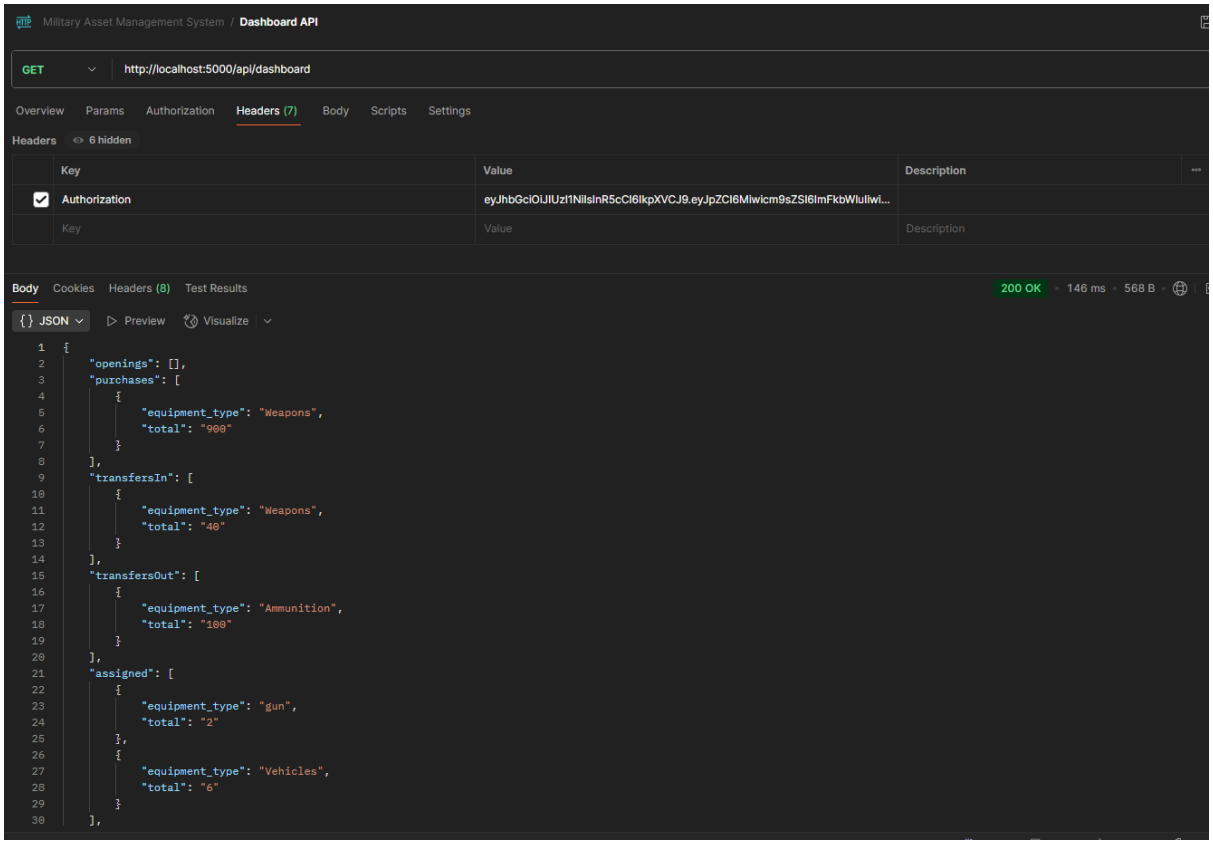
Returns:

{ "token": "JWT_TOKEN_HERE" }



Dashboard API

GET /api/dashboard *(Role-based access)*



Purchases API

POST /api/purchases

{

"base_id": 1,

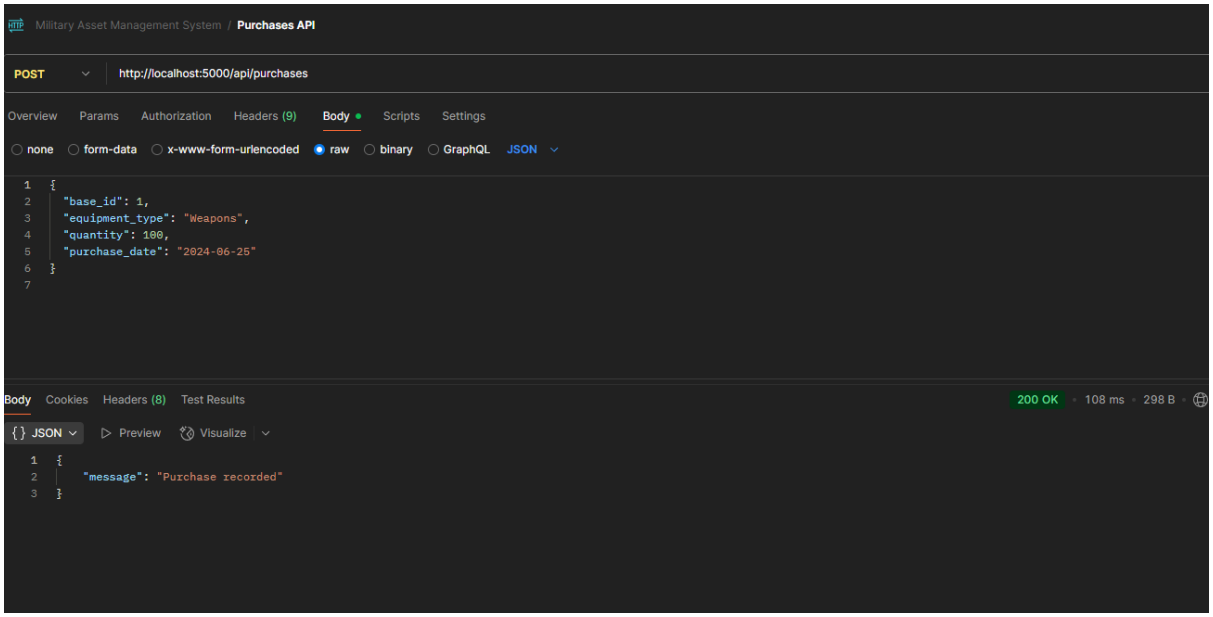
"equipment_type": "Weapons",

"quantity": 100,

"purchase_date": "2024-06-25"

}

GET /api/purchases *(Admin only)*



Transfers API

POST /api/transfers

{

"from_base_id": 1,

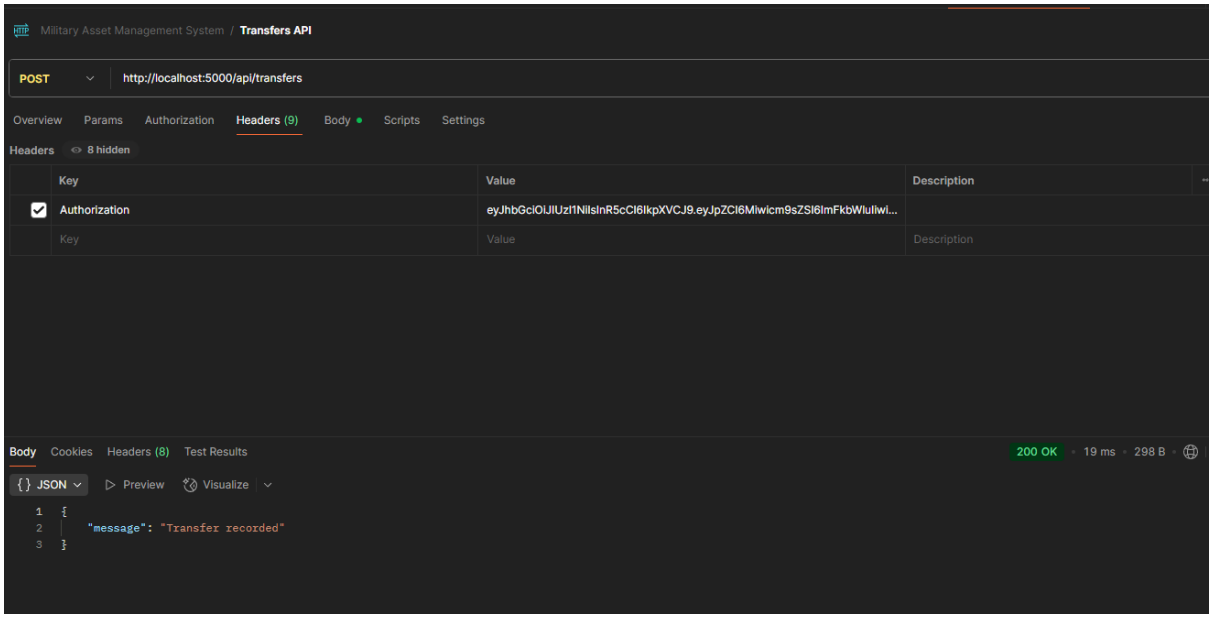
"to_base_id": 2,

"equipment_type": "Ammunition",

"quantity": 50,

"transfer_date": "2024-06-25"

}



Assignments API

POST /api/assignments

{

"base_id": 1,

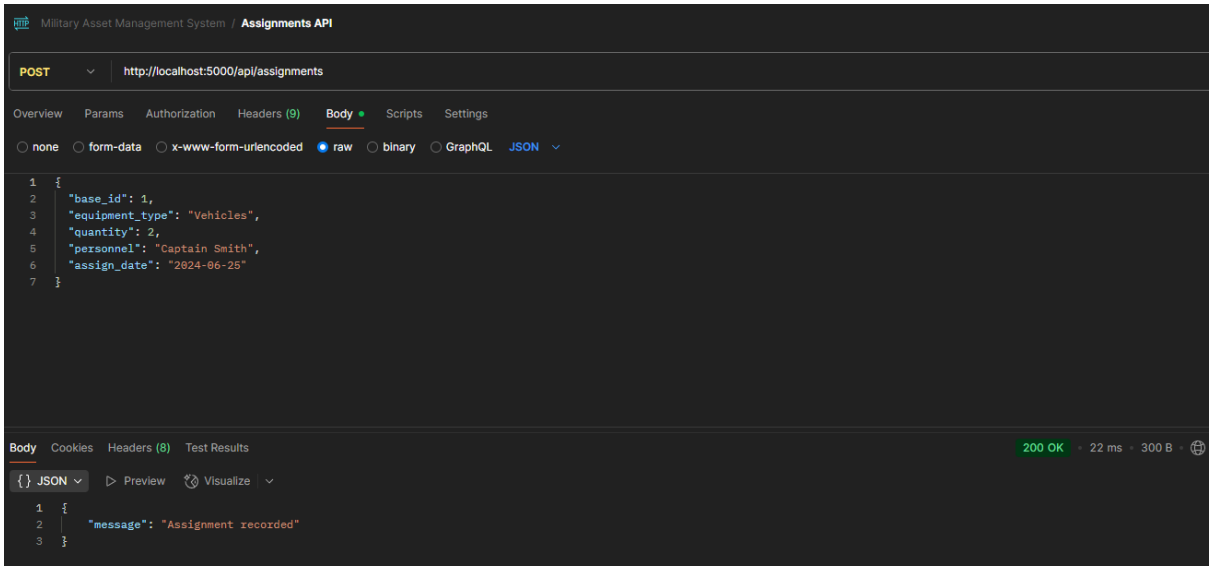
"equipment_type": "Vehicles",

"quantity": 3,

"assigned_to": "Captain Smith",

"assignment_date": "2024-06-25"

}



Expenditures API

POST /api/expenditures

```
{
  "base_id": 1,
  "equipment_type": "Ammunition",
  "quantity": 20,
  "expended_date": "2024-06-25"
}
```

8. Conclusion

This Military Asset Management System provides a secure, scalable, and transparent solution to track critical military resources. The system ensures accountability, enables efficient resource allocation, and provides comprehensive auditing for decision makers. The modular architecture allows future scalability to integrate reporting, analytics, mobile apps, or cloud deployments.

With the clean separation of backend APIs, secured authentication, role-based access control, and detailed logging, the system is production-ready for small to large scale military asset operations. The choice of MySQL ensures stable relational data management while Node.js offers a lightweight server-side execution model suitable for scalable RESTful APIs.

Prepared by: Ashutosh Kumar
Technology Used: Node.js, Express.js, MySQL, REST API, JWT, RBAC, API Logging